DATABASE PROGRESS REPORT

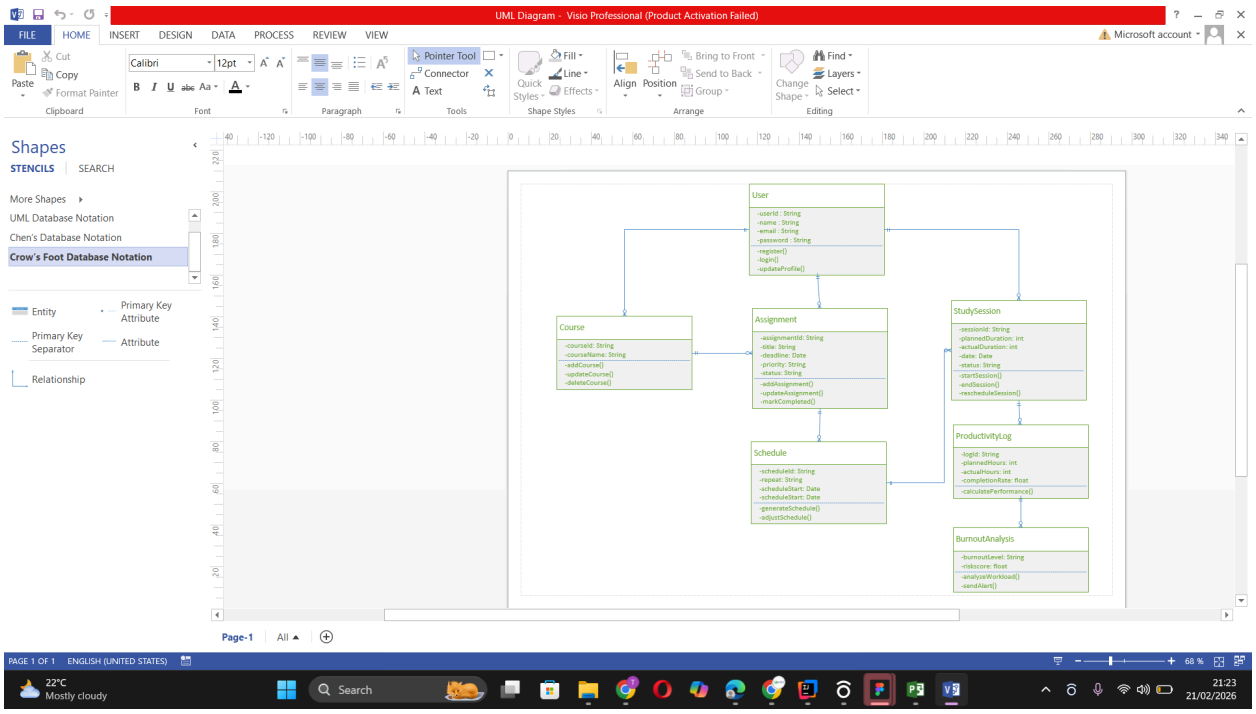
1. Details

- **Project Name:** U-Plan: Intelligent Study Planner for University Students
- **Student Name:** Tunga Greta
- **ID:** 666167

2. Completed Database / Dataset Features

2.1 Database Design

The system uses a NoSQL database (Firebase Firestore) to store and manage application data. However, a relational model was first designed to ensure proper structure and normalization.



Entities Identified

- User
- Course
- Assignment
- StudySession
- ProductivityLog
- BurnoutAnalysis

Relationships

- One User → Many Courses
- One Course → Many Assignments
- One User → Many Study Sessions
- One Study Session → Many Productivity Logs

Primary & Foreign Keys (Conceptual)

- userId (Primary Key in User)
- courseId (Primary Key in Course, FK → userId)
- assignmentId (PK, FK → courseId)
- sessionId (PK, FK → userId)

Normalization

The system design follows Third Normal Form (3NF):

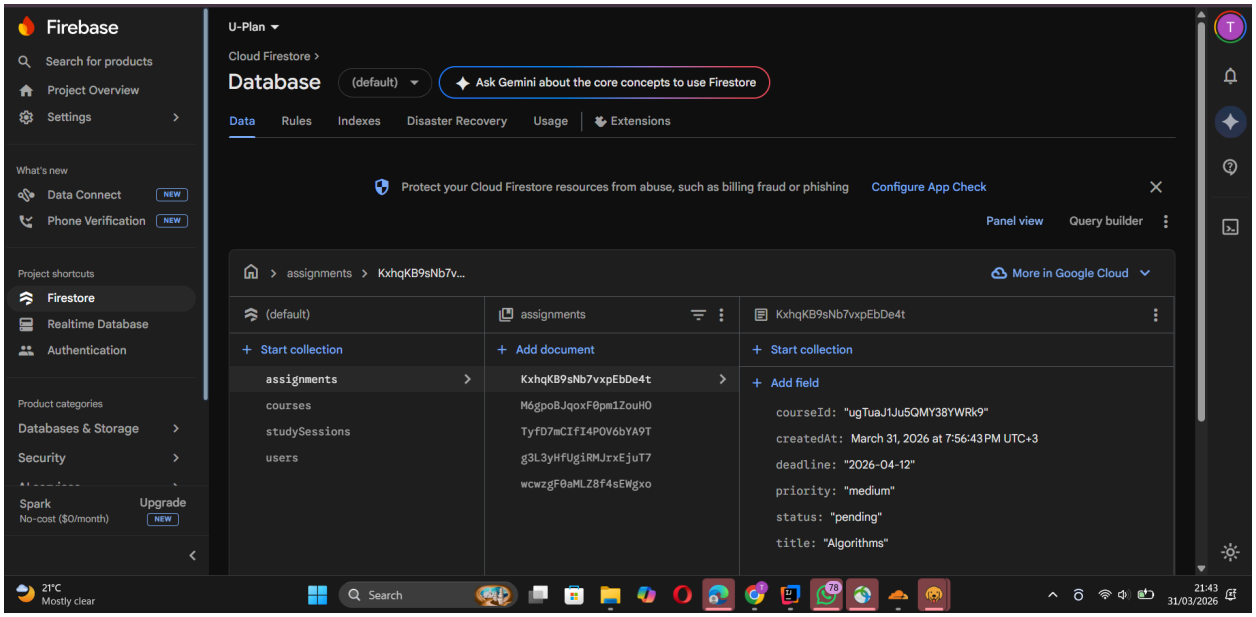
- No repeating groups
- All attributes depend on primary keys
- No transitive dependencies

2.2 Schema Implementation

Since Firestore is schema-less, collections are structured logically as follows:

Collections

- users
- courses
- assignments
- studySessions
- productivityLogs



Constraints (Handled at Application Level)

- Required fields validation (NOT NULL equivalent)
- Unique email for users
- Valid priority values (low, medium, high)
- Default values (e.g., status = “pending”)

Relationships

Implemented using document references (IDs):

- course.userId → links course to user
- assignment.courseId → links assignment to course
- session.userId → links session to user

2.3 Data Management

Sample Data (Firestore Example)

{
"courseId": "gAXQ2AbSDPM4SDFhN7kX",
"userId": "4tHnpqGUfSesaO7dU6NTVSF42Bv1",
"courseName": "Database Systems",
"creditUnits": 3,

}

CRUD Operations Implemented

- **Create:** Add courses, assignments, sessions
- **Read:** Fetch user-specific data
- **Update:** Mark assignments complete, update sessions
- **Delete:** Remove courses and related data

Queries Across Data

- Fetch assignments by course
- Fetch sessions by user
- Aggregate study sessions for analytics

2.4 Indexes & Optimization

- Firestore automatically indexes fields

- Frequently queried fields:
    - userId
    - courseId
    - deadline

Optimization Techniques

- Filtering queries by userId
- Limiting dataset size per request
- Efficient data structuring to avoid nested queries

2.5 Security & Integrity

Data Validation

- Input validation in backend controllers
- Required fields enforced before saving

User Access Control

- Data is scoped per user (userId-based filtering)
- Only authenticated users can access their data

Backup Strategy

- Firebase automatic backups
- Option for manual export of collections

2.6 Advanced Features

Transactions

- Used when updating study sessions to ensure consistency

Business Logic (Equivalent to Stored Procedures)

- Implemented in backend services:

    - Scheduling logic
    - Burnout detection
    - Analytics computation

Triggers

- Not implemented yet (planned for future improvements)

3. Pending / In-Progress Features

- Advanced analytics refinement (consistency score, improved burnout logic)
- Index optimization for large datasets
- Frontend integration with database
- Firebase security rules implementation

4. Challenges & Solutions

Challenge 1: NoSQL vs Relational Requirements

- Firestore does not support traditional joins
- **Solution:** Use references (IDs) and aggregate data in backend

Challenge 2: Authentication Limitations

- Firebase Admin does not support direct login validation
- **Solution:** Simulated login using Firestore user records

Challenge 3: Accurate Study Time Tracking

- Breaks affected total study duration
- **Solution:** Implemented break tracking and adjusted duration calculations

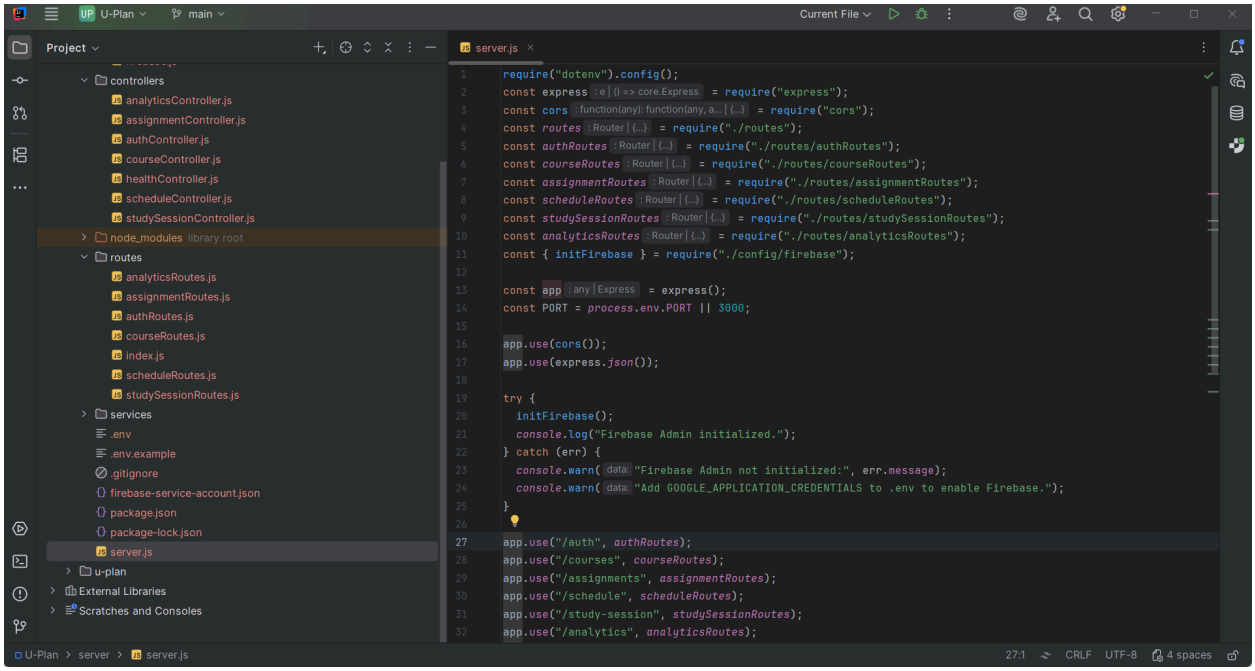
Challenge 4: Scheduling Complexity

- Distributing workload evenly across days

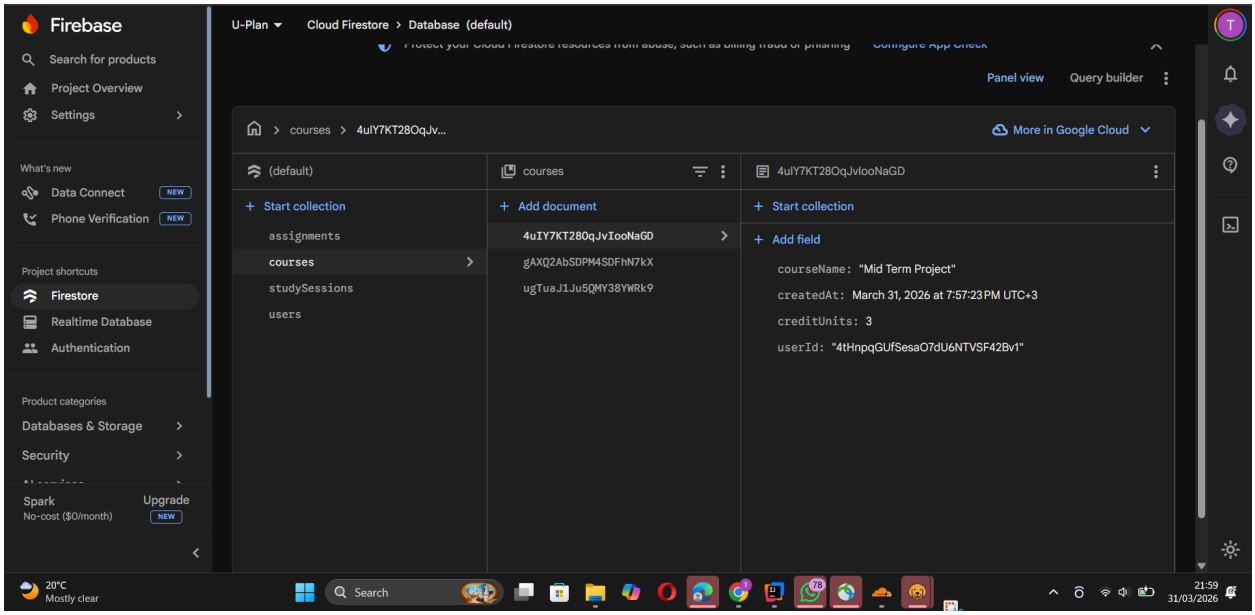
- **Solution:** Implemented a rule-based scheduling algorithm

5. Evidence of Work

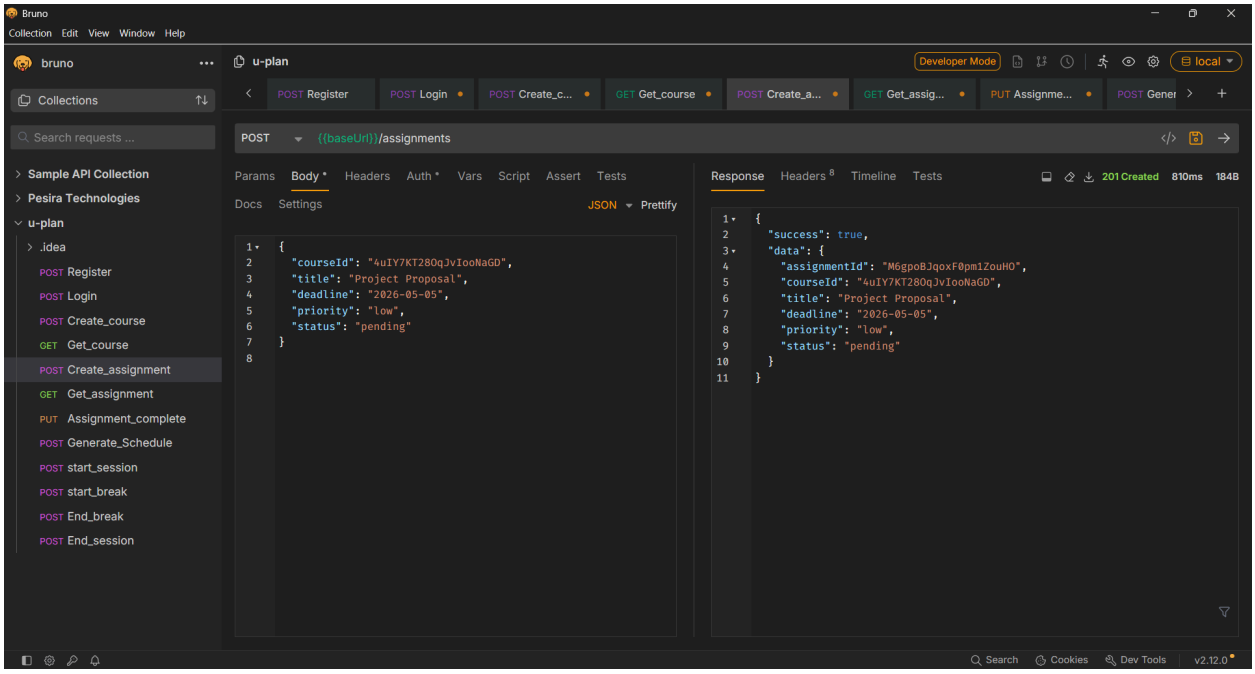
- Backend API implemented using Node.js and Express

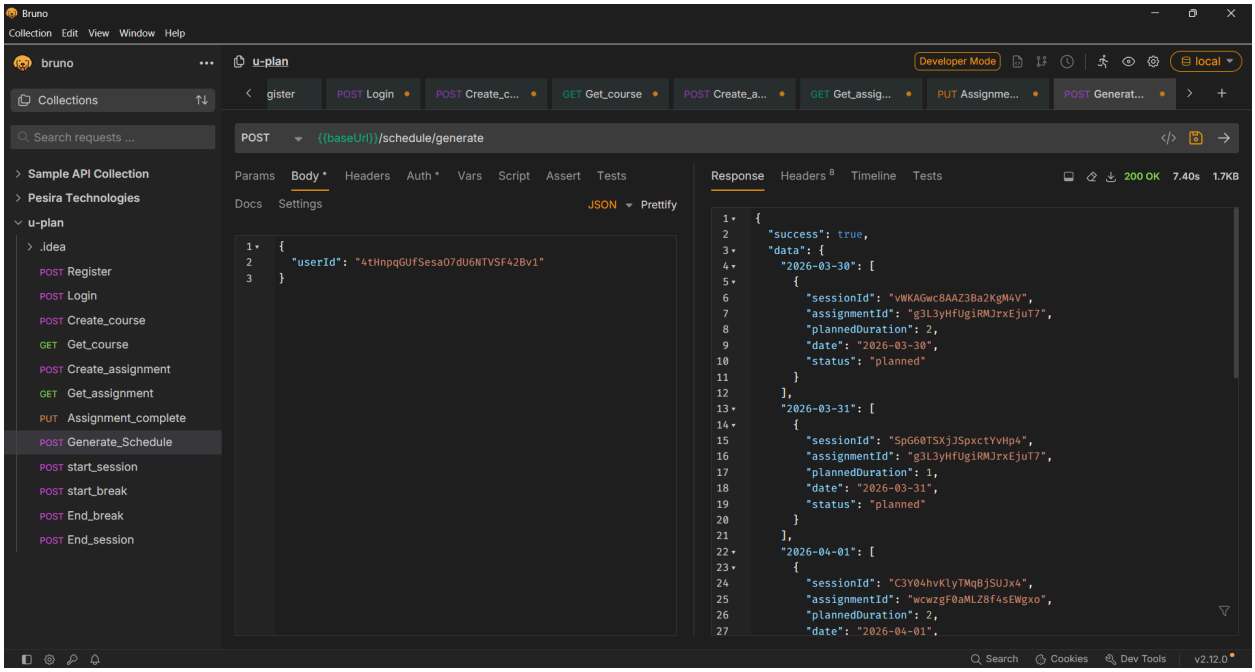


- Firestore collections created and populated



- CRUD operations tested via Bruno





6. Next Steps

- Integrate the frontend with the backend
- Refine analytics and burnout detection
- Strengthen Firebase security rules